

MicroBlaze Debug Module V (MDM V) v1.0

LogiCORE IP Product Guide

Vivado Design Suite

PG428 (v1.0) June 11, 2025



Table of Contents

Chapter 1: Introduction.....	4
Features.....	4
IP Facts.....	5
Chapter 2: Overview.....	6
Navigating Content by Design Process.....	6
Core Overview.....	7
Unsupported Features.....	8
Licensing and Ordering.....	9
Chapter 3: Product Specification.....	10
Resource Utilization.....	10
Standards.....	10
Serial Debug Protocol Description.....	10
Port Descriptions.....	13
Register Space.....	19
Chapter 4: Designing with the Core.....	28
General Design Guidelines.....	28
Clocking.....	30
Resets.....	31
Debug Transport Module Interface Description.....	31
External Trace Connection.....	32
Chapter 5: Design Flow Steps.....	34
Customizing and Generating the Core.....	34
Constraining the Core.....	38
Simulation.....	40
Synthesis and Implementation.....	40
Appendix A: Verification, Compliance, and Interoperability.....	41



Appendix B: Upgrading..... 42

Appendix C: Debugging.....43

 Finding Help with AMD Adaptive Computing Solutions.....43

 Debug Tools..... 44

 Simulation Debug.....45

 Hardware Debug..... 45

 Interface Debug..... 46

Appendix D: Application Software Development..... 47

 Device Drivers..... 47

Appendix E: Additional Resources and Legal Notices.....48

 Finding Additional Documentation..... 48

 Support Resources..... 49

 References.....49

 Revision History..... 50

 Please Read: Important Legal Notices..... 50

Introduction

This document provides the design specification for the MicroBlaze™ Debug Module V (MDM V) core which enables JTAG-based debugging of one or more MicroBlaze V processors. The MDM V core is added separately in the AMD Vivado™ Design Suite and connected to the MicroBlaze V processors to be debugged.

Features

- Support for JTAG-based software debug tools
- Support for debugging up to 32 MicroBlaze V processors
- Support for synchronized control of multiple MicroBlaze V processors
- Support for a JTAG-based UART using a configurable AXI4-Lite interface
- Based on Boundary Scan (BSCAN) logic in AMD devices
- Direct access to memory with a configurable AXI4 master interface and LMB interfaces
- Configurable software access to debug functionality through the AXI4-Lite interface
- Trace function to control RISC-V N-Trace in connected MicroBlaze V processors
- External trace function to funnel trace from connected MicroBlaze V cores to an external interface using RISC-V N-Trace Funnel and PIB Sink
- Connection to Debug Bridge through external BSCAN to support Xilinx Virtual Cable (XVC)

IP Facts

AMD LogiCORE™ IP Facts Table	
Core Specifics	
Supported Device Family ¹	AMD UltraScale+™, AMD UltraScale™, AMD Zynq™ 7000 SoC, 7 series, AMD Versal™ adaptive SoC
Supported User Interfaces	AXI4, AXI4-Lite
Resources	Performance and Resource Utilization web page
Provided with Core	
Design Files	RTL
Example Design	Not Provided
Test Bench	Not Provided
Constraints File	Not Provided
Simulation Model	VHDL Behavioral
Supported S/W Driver	Standalone
Tested Design Flows ²	
Design Entry	AMD Vivado™ Design Suite
Simulation	For supported simulators, see the <i>Vivado Design Suite User Guide: Release Notes, Installation, and Licensing</i> (UG973).
Synthesis	Vivado Synthesis
Support	
Release Notes and Known Issues	Master Answer Record: 36210
All Vivado IP Change Logs	Master Vivado IP Change Logs: 72775
Support web page	

Notes:

1. For a complete list of supported devices, see the AMD Vivado™ IP catalog.
2. For the supported versions of the tools, see the *Vivado Design Suite User Guide: Release Notes, Installation, and Licensing* ([UG973](#)).

Overview

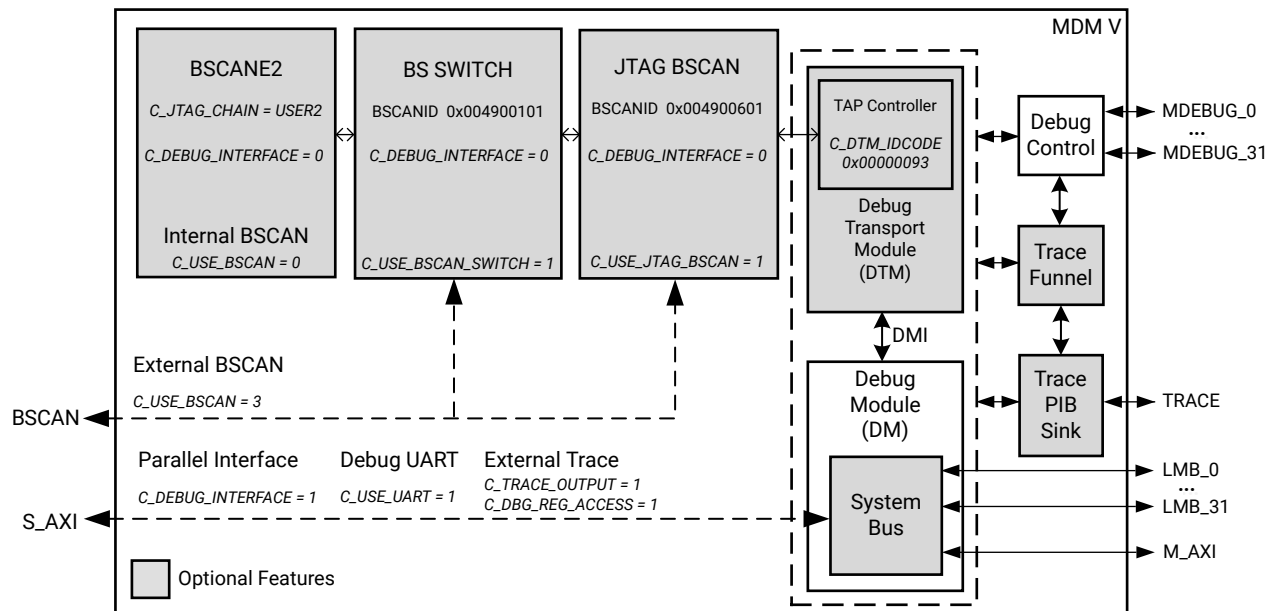
Navigating Content by Design Process

AMD Adaptive Computing documentation is organized around a set of standard design processes to help you find relevant content for your current development task. You can access the AMD Versal™ adaptive SoC design processes on the [Design Hubs](#) page. You can also use the [Design Flow Assistant](#) to better understand the design flows and find content that is specific to your intended design needs. This document covers the following design processes:

- **Embedded Software Development:** Creating the software platform from the hardware platform and developing the application code using the embedded CPU. Also covers XRT and Graph APIs. Topics in this document that apply to this design process include:
 - [Customizing and Generating the Core](#)
 - [Parameter Values](#)
 - [Appendix D: Application Software Development](#)
- **Hardware, IP, and Platform Development:** Creating the PL IP blocks for the hardware platform, creating PL kernels, functional simulation, and evaluating the AMD Vivado™ timing, resource use, and power closure. Also involves developing the hardware platform for system integration. Topics in this document that apply to this design process include:
 - [Port Descriptions](#)
 - [Clocking](#)
 - [Resets](#)
 - [Customizing and Generating the Core](#)
- **System Integration and Validation:** Integrating and validating the system functional performance, including timing, resource use, and power closure. Topics in this document that apply to this design process include:
 - [Resource Utilization](#)
 - [External Trace Connection](#)

Core Overview

The MDM V core block diagram is shown in the following figure.



X24565-050725

Feature Summary

- Enables JTAG-based debugging of one or more MicroBlaze™ V processors.
- Instantiates one BSCAN primitive, or allows an external BSCAN to be used. In devices that contain more than one BSCAN primitive, the MDM V core uses the USER2 BSCAN by default.
- External BSCAN also supports connection to the Debug Bridge AMD LogiCORE™ IP, to use the Xilinx Virtual Cable (XVC) for debugging over non-JTAG interfaces.

In AMD Versal™ devices the external BSCAN is normally hidden and the Control, Interfaces and Processing System (CIPS) BSCAN is automatically connected by the AMD Vivado™ Design Suite, which provides a transparent functionality equivalent to other devices. However, it is possible to manually connect the CIPS BSCAN to the external BSCAN, if necessary. For more information on CIPS, see the *Control, Interface and Processing System LogiCORE IP Product Guide* (PG352).

- Includes a UART with a configurable slave bus interface, which can be configured for an AXI4-Lite interconnect. The UART TX and RX data is transferred over the device JTAG port to and from the System Debugger (XSDB) tool. The UART behaves in a manner similar to the LogiCORE IP AXI (UART) Lite core.

- Provides a configurable AXI4 master port and LMB interfaces for direct access to memory from JTAG or parallel debug, using the RISC-V System Bus function. This allows fast program download, as well as transparent memory access when the connected processors are executing.
- Allows software to control debug and observe debug status through the AXI4-Lite slave interface with parallel debug enabled.
- Includes support for control of RISC-V N-Trace (Nexus-based Trace) components in each connected MicroBlaze V processor through the debug interface, using the RISC-V System Bus function.
- Includes support for external trace interfaces to funnel and transmit MicroBlaze V program trace. Program trace from connected MicroBlaze V processors can be directly output on an external interface using a RISC-V Trace Funnel and Trace PIB Sink.
- Supports MicroBlaze V parallel debug access, which can be used instead of serial debug to provide faster direct access to MicroBlaze V debug registers, and to improve timing compared to serial debug.

In general, it is recommended to only use this feature when software debug through JTAG is not required, because otherwise, the MDM V requires additional logic.

Unsupported Features

The following optional features of the RISC-V External Debug Support, Version 1.0, standard are not supported in the core:

- Register access without halting
- A running hart executing short sequence of instructions
- RISC-V hart halted when a trigger matches an instruction opcode
- Abstract access to non-GPR hart registers
- Register number post-increment (`aarpostincrement` bit)
- Quick Access abstract command
- Access Memory abstract command
- Hart info (`hartinfo`) register
- Abstract Command Autoexec (`abstractauto`) register
- Debug Scratch Register 0 and 1 (`dscratch0`, `dscratch1`)
- Trigger Control (`tcontrol`) register
- Trigger Extra (`textra32`) register

Some registers are also implicitly excluded because only a maximum of 32 harts are supported.

Licensing and Ordering

This AMD LogiCORE™ IP module is provided at no additional cost with the AMD Vivado™ Design Suite under the terms of the [End User License](#).

Information about other AMD LogiCORE™ IP modules is available at the [Intellectual Property](#) page. For information about pricing and availability of other AMD LogiCORE IP modules and tools, contact your [local sales representative](#).

Product Specification

Resource Utilization

For details about resource utilization, visit [Performance and Resource Utilization](#).

Standards

The MDM V core adheres to the AMBA® AXI4 and AXI4-Lite Interface standard (see AMBA AXI and ACE Protocol Specification ([ARM IH0022E](#))).

The MDM V core implements all mandatory features of the RISC-V External Debug Support, Version 1.0.

The MDM V core implements external trace according to the RISC-V N-Trace (Nexus-based Trace) Specification and the RISC-V Trace Control Interface Specification.

Serial Debug Protocol Description

The MDM V has an internal TAP Controller that conforms to the RISC-V DTM. To generate the TAP Controller signals from BSCAN, an internal version of the BSCAN to JTAG Converter LogiCORE IP is utilized. See *BSCAN to JTAG Converter LogiCORE IP Product Guide* ([PG365](#)) for more information.

The BSCAN to JTAG protocol controls the internal TAP Controller by shifting in data on `bscan_ext_tdi`. Each transition of the internal TAP Controller requires two `bscan_ext_tck` cycles, except when in the SHIFT-DR or SHIFT-IR state, where one `bscan_ext_tck` cycle is required:

- Shift in 00 to set the internal TAP Controller TMS to 0 and perform one internal TCK cycle
- Shift in 01 to set internal TAP Controller TMS to 1 and perform one internal TCK cycle

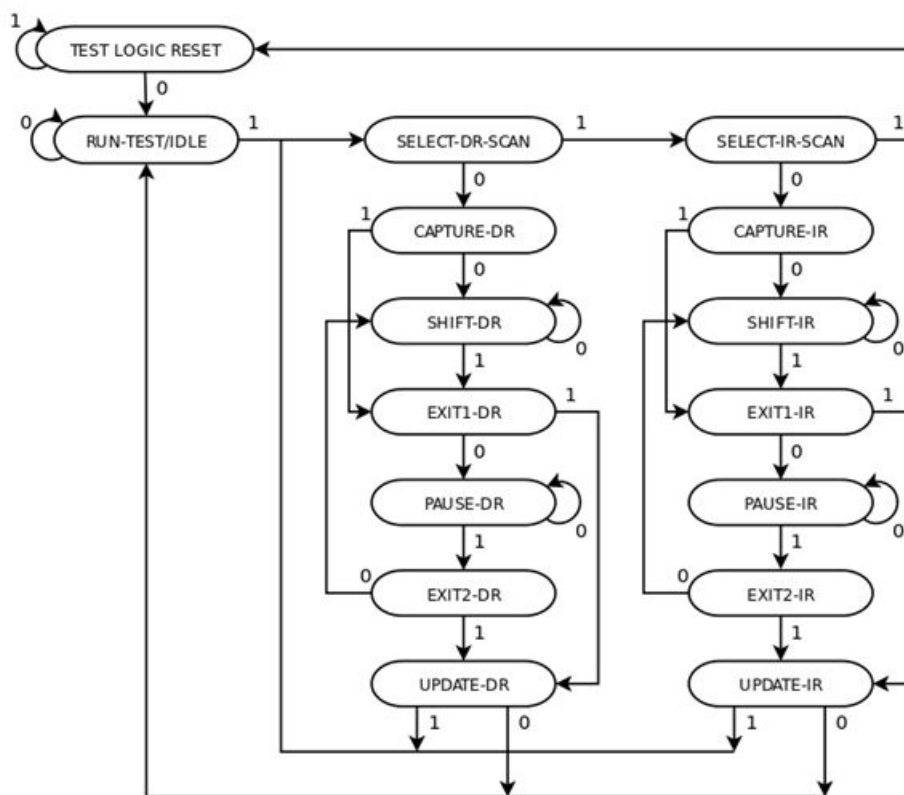
- To leave the internal TAP Controller SHIFT-DR or SHIFT-IR state, BSCAN must set `bscan_ext_tms` to 1, which transitions to EXIT1-DR

During accesses to the internal TAP Controller, BSCAN need not go to UPDATE-DR, but can transition from SHIFT-DR via EXIT1-DR, PAUSE-DR, EXIT2-DR back to SHIFT-DR.

Additionally, the following BSCAN signals must be set according to the protocol:

- `bscan_ext_shift` should be set in the state SHIFT-DR
- `bscan_ext_update` should be set in the state UPDATE-DR
- `bscan_ext_capture` should be set in the state CAPTURE-DR
- `bscan_ext_sel` should be set to 1 when transfers are directed to MDM RISC-V
- `bscan_ext_drck` is not used, but should be equal to `bscan_ext_tck` in the states CAPTURE-DR and SHIFT-DR

Figure 1: TAP Controller State Diagram



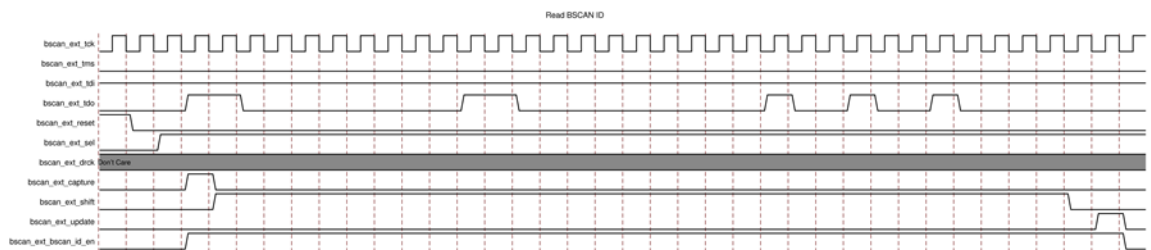
Initial Read of BSCANID

Initial read of BSCANID must be done after every BSCAN reset activation.

- Set `bscan_ext_reset` = 1 for 1 `bscan_ext_tck` cycle

- Set `bscan_ext_sel = 1` and `bscan_ext_bscanid_en = 1` when used (the parameter `C_BSCANID` is non-zero)
- Set `bscan_ext_capture = 1` for 1 `bscan_ext_tck` cycle
- Set `bscan_ext_shift = 1` for 32 `bscan_ext_tck` cycles
- Set `bscan_ext_update = 1` for 1 `bscan_ext_tck` cycle
- Set `bscan_ext_bscanid_en = 0` when used
- Verify that `0x04900601` is received on `bscan_ext_tdo` during shift

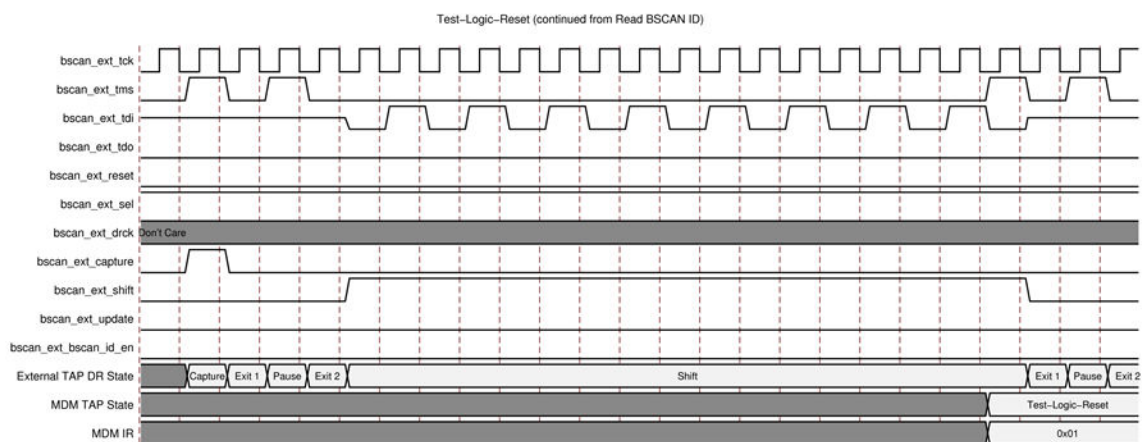
Figure 2: Initial Read of BSCANID



Go to Test-Logic Reset

Ensure that the MDM RISC-V internal TAP Controller is in Test-Logic-Reset. This should be done after the initial read of BSCANID, or otherwise after every BSCAN reset activation. It is done by shifting in `0101010101010101` on `bscan_ext_tdi` to set the internal TMS to 1 for 8 internal TCK cycles.

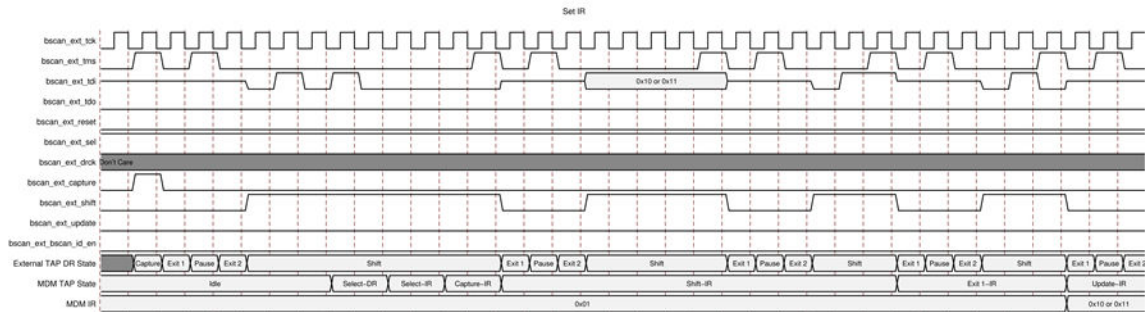
Figure 3: Go to Test-Logic-Reset



Set Instruction Register and Shift Data

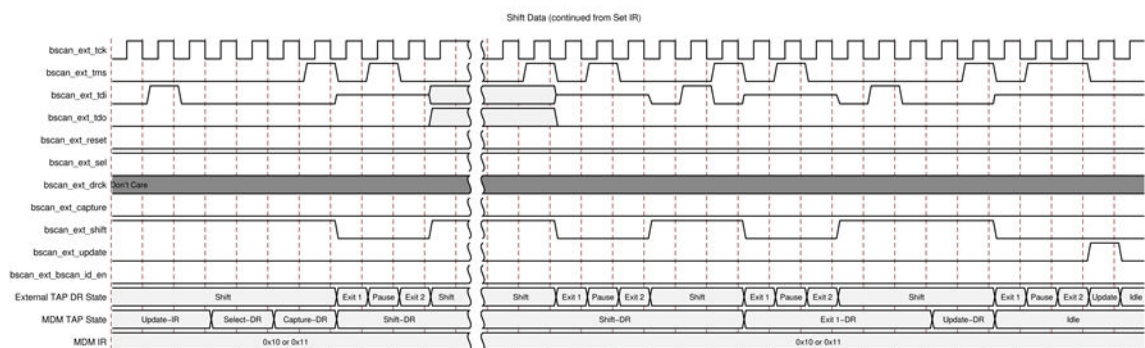
Set the internal TAP Controller Instruction Register (IR) by transitioning to Shift-IR from Test-Logic-Reset with 010100000 on `bscan_ext_tdi`, and then shifting in the 5-bit IR value (0x10 for dtmcs or 0x11 for dmi) on `bscan_ext_tdi`. Then transition to Update-IR from Shift-IR with 011 010 on `bscan_ext_tdi`.

Figure 4: Set Instruction Register



Shift data to the internal TAP Controller by transitioning to Shift-DR from Update-IR via Capture-DR with 0100000 on `tdi`, and then shifting in the new 32-bit register value on `bscan_ext_tdi`, simultaneously with capturing the old register value on `bscan_ext_tdo`. Then transition from Shift-DR to Update-DR with 010 on `bscan_ext_tdi`, and finally transition back to Idle with 01000 on `bscan_ext_tdi`.

Figure 5: Shift Data



Port Descriptions

The I/O signals for the MDM V core are listed and described in the following tables.

Table 1: System Signals

Signal Name	Interface	I/O	Initial State	Description
Interrupt		O	0	Interrupt from UART
Debug_SYS_Rst		O	0	Debug system reset

MicroBlaze V Serial Debug Interface Signals

Table 2: MicroBlaze V Serial Debug I/F Signals (n = 0–31, C_DEBUG_INTERFACE = 0)

Signal Name	Interface	I/O	Initial State	Description
Dbg_Disable_n	MBDEBUG_n	O	1	MicroBlaze debug disable
Dbg_Clk_n		O	0	MicroBlaze Debug Clock
Dbg_TDI_n		O	0	MicroBlaze Debug TDI
Dbg_TDO_n		I	-	MicroBlaze debug TDO
Dbg_Reg_En_n		O	0	MicroBlaze debug register enable
Dbg_Capture_n		O	0	MicroBlaze debug capture
Dbg_Shift_n		O	0	MicroBlaze debug shift
Dbg_Update_n		O	0	MicroBlaze debug update

MicroBlaze V Parallel Debug Interface Signals

Table 3: MicroBlaze V Parallel Debug I/F Signals (n = 0-31)

Signal Name	Interface	I/O	Initial State	Description
Dbg_AWADDR_n	MBDEBUG_n MBDEBUG_AXI_n	O	0	MicroBlaze debug write address
Dbg_AWVALID_n		O	0	MicroBlaze debug write address valid
Dbg_AWREADY_n		I	-	MicroBlaze debug write address ready
Dbg_WDATA_n		O	0	MicroBlaze debug write data
Dbg_WVALID_n		O	0	MicroBlaze debug write data valid
Dbg_WREADY_n		I	-	MicroBlaze debug write data ready
Dbg_BRESP_n		I	-	MicroBlaze debug write response
Dbg_BVALID_n		I	-	MicroBlaze debug write response valid
Dbg_BREADY_n		O	0	MicroBlaze debug write response ready
Dbg_ARADDR_n		O	0	MicroBlaze debug read address
Dbg_ARVALID_n		O	0	MicroBlaze debug read address valid
Dbg_ARREADY_n		I	-	MicroBlaze debug read address ready
Dbg_RDATA_n		I	-	MicroBlaze debug read data
Dbg_RRESP_n		I	-	MicroBlaze debug read data response
Dbg_RVALID_n		I	-	MicroBlaze debug read data valid
Dbg_RREADY_n		O	0	MicroBlaze debug read data ready

Notes:

- The signals are enabled when using:
 - Parallel debug (C_DEBUG_INTERFACE>0)
 - Embedded trace without memory access from debug (C_TRACE_OUTPUT=4 and C_DBG_MEM_ACCESS=0)
 - External trace without debug register access from AXI or memory access from debug (C_TRACE_OUTPUT=1 and C_DBG_REG_ACCESS=0 and C_DBG_MEM_ACCESS=0)

MicroBlaze V Other Debug Interface Signals

Table 4: MicroBlaze V Other Debug I/F Signals (n = 0-31, C_DEBUG_INTERFACE ≠ 2)

Signal Name	Interface	I/O	Initial State	Description
Dbg_Disable_n	MBDEBUG_n	O	1	MicroBlaze debug disable
Dbg_Rst_n		O	0	MicroBlaze debug reset
Dbg_TrClk_n		O	0	MicroBlaze debug trace clock
Dbg_TrData_n		I	-	MicroBlaze debug trace data
Dbg_TrReady_n		O	0	MicroBlaze debug trace ready
Dbg_TrValid_n		I	-	MicroBlaze debug trace valid

AXI4-Lite Slave Interface Signals

Table 5: AXI4-Lite Slave Interface Signals (C_DBG_REG_ACCESS = 1 or C_USE_UART = 1)

Signal Name	Interface	I/O	Initial State	Description
S_AXI_ACLK	S_AXI	I	-	AXI Clock
S_AXI_ARESETN		I	-	AXI Reset, active-Low
S_AXI_AWADDR[C_S_AXI_ADDR_WIDTH-1:0]		I	-	Write Address
S_AXI_AWVALID		I	-	Write Address Valid
S_AXI_AWREADY		O	0	Write Address Ready
S_AXI_WDATA[C_S_AXI_DATA_WIDTH-1:0]		I	-	Write Data
S_AXI_WSTB[C_S_AXI_DATA_WIDTH/8-1:0]		I	-	Write Strobes
S_AXI_WVALID		I	-	Write Valid
S_AXI_WREADY		O	0	Write Ready
S_AXI_BRESP[1:0]		O	0x0	Write Response
S_AXI_BVALID		O	0	Write Response Valid
S_AXI_BREADY		I	-	Write Response Ready
S_AXI_ARADDR[C_S_AXI_ADDR_WIDTH-1:0]		I	-	Read Address
S_AXI_ARVALID		I	-	Read Address Valid
S_AXI_ARREADY		O	0	Read Address Ready
S_AXI_RDATA[C_S_AXI_DATA_WIDTH-1:0]		I	-	Read Data
S_AXI_RRESP[1:0]		O	0x0	Read Response
S_AXI_RVALID		O	0	Read Valid
S_AXI_RREADY		I	-	Read Ready

AXI4 Master Interface Signals

Table 6: AXI4 Master Interface Signals (C_DBG_MEM_ACCESS = 1)

Signal Name	Interface	I/O	Initial State	Description
M_AXI_ACLK	M_AXI	I	-	AXI Clock
M_AXI_ARESETN		I	-	AXI Reset, active-Low
M_AXI_AWID[C_M_AXI_THREAD_ID_WIDTH-1:0]		O	0x0	Write Address ID
M_AXI_AWADDR[C_M_AXI_ADDR_WIDTH-1:0]		O	0x0	Write Address
M_AXI_AWLEN[7:0]		O	0x0	Write Address Length
M_AXI_AWSIZE[2:0]		O	0x2	Write Address Size
M_AXI_AWBURST[1:0]		O	0x1	Write Address Burst
M_AXI_AWLOCK		O	0	Write Address Lock
M_AXI_AWCACHE[3:0]		O	0x3	Write Address Cache
M_AXI_AWPROT[2:0]		O	0x2	Write Address Protection
M_AXI_AWQOS[3:0]		O	0x0	Write Address QoS
M_AXI_AWVALID		O	0	Write Address Valid
M_AXI_AWREADY		I	-	Write Address Ready
M_AXI_WDATA[C_M_AXI_DATA_WIDTH-1:0]		O	0x0	Write Data
M_AXI_WSTB[C_M_AXI_DATA_WIDTH/8-1:0]		O	0x0	Write Strokes
M_AXI_WVALID		O	0	Write Valid
M_AXI_WREADY		I	-	Write Ready
M_AXI_BRESP[1:0]		I	-	Write Response
M_AXI_BID[C_M_AXI_THREAD_ID_WIDTH-1:0]		I	-	Write Response ID
M_AXI_BVALID		I	-	Write Response Valid
M_AXI_BREADY		O	0	Write Response Ready
M_AXI_ARID[C_M_AXI_THREAD_ID_WIDTH-1:0]		O	0x0	Read Address ID
M_AXI_ARADDR[C_M_AXI_ADDR_WIDTH-1:0]		O	0x0	Read Address
M_AXI_ARLEN[7:0]		O	0x0	Read Address Length
M_AXI_ARSIZE[2:0]		O	0x0	Read Address Size
M_AXI_ARBURST[1:0]		O	0x1	Read Address Burst
M_AXI_ARLOCK		O	0	Read Address Lock
M_AXI_ARCACHE[3:0]		O	0x3	Read Address Cache
M_AXI_ARPROT[2:0]		O	0x2	Read Address Protection
M_AXI_ARQOS[3:0]		O	0x0	Read Address QoS
M_AXI_ARVALID		O	-	Read Address Valid
M_AXI_ARREADY		I	0	Read Address Ready
M_AXI_RID[C_M_AXI_THREAD_ID_WIDTH-1:0]		I	-	Read ID
M_AXI_RDATA[C_M_AXI_DATA_WIDTH-1:0]		I	-	Read Data
M_AXI_RRESP[1:0]		I	-	Read Response
M_AXI_RLAST		I	-	Read Last
M_AXI_RVALID		I	-	Read Valid
M_AXI_RREADY		O	0	Read Ready

External BSCAN Interface Signals

Table 7: External BSCAN Interface Signals (C_USE_BSCAN = 2 or 4)

Signal Name	Interface	I/O	Initial State	Description
bscan_ext_tdi	BSCAN	I	-	Connection to external BSCAN or Debug Bridge
bscan_ext_reset		I	-	
bscan_ext_shift		I	-	
bscan_ext_update		I	-	
bscan_ext_capture		I	-	
bscan_ext_sel		I	-	
bscan_ext_drck		I	-	
bscan_ext_tdo		O	0	
bscan_ext_bscanid_en ¹		I	-	
bscan_ext_tck		I	0	

Notes:

1. Only available when parameter C_BSCANID is not equal to 0.

External Trace Output

Table 8: External Trace Output (C_TRACE_OUTPUT = 1)

Signal Name	Interface	I/O	Initial State	Description
TRACE_DATA[C_TRACE_DATA_WIDTH-1:0]	TRACE	O	-	Connection to an external trace connector defined by the RISC-V Trace Connectors Specification.
TRACE_CTL		O	-	
TRACE_CLK ¹		I	-	
TRACE_CLK_OUT ²		O	-	

Notes:

1. The nominal frequency of TRACE_CLK is 200 MHz. If another clock frequency is used, the parameter C_TRACE_CLK_FREQ_HZ is set from the connected input clock (by propagation in AMD Vivado™ IP integrator) if possible, but must otherwise be manually changed accordingly.
2. The frequency of TRACE_CLK_OUT is TRACE_CLK divided by 2, nominally 100 MHz with a 90° phase shift, to create a sample point at a stable point of the outputs. The phase shift can be adjusted manually with the parameter C_TRACE_CLK_OUT_PHASE if necessary.

Register Space

In the following tables, Access is indicated by:

- **R:** Read-only
- **W:** Write-only
- **R/W:** Read/Write

AXI Slave Register Space

The following table describes the MDM V core registers accessible through the AXI4-Lite slave interface.

Table 9: MDM V Core AXI4-Lite Slave Registers

Register Name	Size (bits)	Address Offset	Access	Description
JTAG UART Registers (C_USE_UART = 1)				
UART_RX_FIFO	8	0x00	R	JTAG UART receive data
UART_TX_FIFO	8	0x04	W	JTAG UART transmit data
UART_STATUS	8	0x08	R	JTAG UART status
UART_CTRL	8	0x0C	W	JTAG UART control
Trace Funnel Registers (C_DBG_REG_ACCESS = 1, C_TRACE_OUTPUT = 1, C_MB_DBG_PORTS > 1)				
trFunnelControl	4	0x1000	R/W	Trace funnel control register
trFunnelImpl	12	0x1004	R	Trace funnel implementation register
trFunnelDisInput	16	0x1008	R	Disable individual funnel inputs
Trace PIB Sink Registers (C_DBG_REG_ACCESS = 1, C_TRACE_OUTPUT = 1)				
trPibControl	32	0x2000	R/W	PIB sink control register
trPibImpl	12	0x2004	R	Trace PIB implementation register

The JTAG UART registers are identical to the AXI UART Lite registers (see the *AXI UART Lite LogiCORE IP Product Guide* ([PG142](#))), except that the Status register bits 5–7 (OverrunError, Frame Error, Parity Error) are never set, and the Control register bit 2 is not reserved.

The Trace Funnel Registers and Trace PIB Sink Registers can either be accessed from the AXI Slave interface (C_DBG_REG_ACCESS = 1) or from the RISC-V System Bus (C_DBG_REG_ACCESS = 0 and C_DBG_MEM_ACCESS = 0). The trFunnelDisInput register is optional, and not implemented in the MDM V.

The MDM V core always responds to accesses within the defined address space. When Parallel Debug Register Access is enabled, the defined address space is 0x000–0x1FF; and when JTAG UART is enabled it is 0x0–0xF. For any unused addresses, write requests are ignored and read requests return zero data.

UART Receive FIFO Register (UART_RX_FIFO)

This 16-entry-deep FIFO contains data received by the UART from JTAG. The FIFO bit definitions are shown in the following table. Reading this register results in reading the data word from the top of the FIFO. When a read request is issued to an empty FIFO, a bus error (SLVERR) is generated and the result is undefined. The register is a read-only register. Issuing a write request to the receive data FIFO does nothing but generates a successful write acknowledgment. The following table shows the location for data on the AXI slave interface. The register is only implemented if C_USE_UART is set to 1.

Reserved				UART_RX			
31		8	7			0	

Table 10: UART Receive FIFO Register Bit Definitions

Bits	Name	Access	Reset Value	Description
31:8	-	R	0	Reserved
7:0	UART_RX	R	0	UART Receive Data

UART Transmit FIFO Register (UART_TX_FIFO)

This 16-entry-deep FIFO contains data to be output by the UART using JTAG. The FIFO bit definitions are shown in the following table. Data to be transmitted is written into this register. When a write request is issued while the FIFO is full, a bus error (SLVERR) is generated and the data is not written into the FIFO. This is a write-only location. Issuing a read request to the transmit data FIFO generates the read acknowledgment with zero data. The following table shows the location for data on the AXI interface. The register is only implemented if C_USE_UART is set to 1.

Reserved				UART_TX			
31		8	7			0	

Table 11: UART Transmit FIFO Register Bit Definitions

Bits	Name	Access	Reset Value	Description
31:8	-	R	0	Reserved
7:0	UART_TX	R	0	UART Transmit Data

UART Status Register (UART_STATUS)

The status register contains the status of the receive and transmit data FIFOs, and when interrupt is enabled. This is a read only register. If a write request is issued to the status register it will do nothing but generate write acknowledgment. Bit assignment in the register is described in the following tables. The register is only implemented if C_USE_UART is set to 1.

Reserved			UART_STATUS	
31	5	4		0

Table 12: UART Status Register Bit Definitions

Bits	Name	Access	Reset Value	Description
31-5	Reserved	N/A	0	Reserved
4	Interrupt Enabled	R	0	Indicates that interrupt is enabled. 0 = Interrupt is disabled 1 = Interrupt is enabled
3	TX FIFO Full	R	0	Indicates if the transmit FIFO is full. 0 = Transmit FIFO is not full 1 = Transmit FIFO is full
2	TX FIFO Empty	R	1	Indicates if the transmit FIFO is empty. 0 = Transmit FIFO is not empty 1 = Transmit FIFO is empty
1	RX FIFO Full	R	0	Indicates if the receive FIFO is full. 0 = Receive FIFO is not full 1 = Receive FIFO is full
0	RX FIFO Valid Data	R	0	Indicates if the receive FIFO has valid data. 0 = Receive FIFO is empty 1 = Receive FIFO has valid data

UART Control Register (UART_CTRL)

The control register contains the enable interrupt bit and reset for the receive and transmit data FIFO. This is write only register. Issuing a read request to the control register generates the read acknowledgment with zero data. Bit assignment in the register is described in the following tables. The register is only implemented if C_USE_UART is set to 1.

Reserved			UART_CTRL	
31	5	4		0

Table 13: UART Control Register Bit Definitions

Bits	Name	Access	Reset Value	Description
31-5	Reserved	N/A	0	Reserved

Table 13: UART Control Register Bit Definitions (cont'd)

Bits	Name	Access	Reset Value	Description
4	Interrupt Enabled	W	0	Enable interrupt for the MDM JTAG UART 0 = Disable interrupt signal 1 = Enable interrupt signal
3	Reserved	N/A	0	Reserved
2	Clear EXT_BRK signal	W	0	Clear the EXT_BRK signal set by JTAG 0 = Do nothing 1 = Clear the signal
1	Reset RX FIFO	W	1	Reset/clear the receive FIFO Writing a 1 to this bit position clears the receive FIFO 0 = Do nothing 1 = Clear the receive FIFO
0	Reset TX FIFO	W	1	Reset/clear the transmit FIFO Writing a 1 to this bit position clears the transmit FIFO 0 = Do nothing 1 = Clear the transmit FIFO

Trace Funnel Control Register

The Trace Funnel Control Register is only implemented when external trace is enabled (C_TRACE_OUTPUT = 1) and when more than one processor is connected to the MDM V (C_MB_DBG_PORTS > 1).

Reserved		Empty	-	Enable	Active
31	4	3	2	1	0

Table 14: Trace Funnel Control Register Bit Definitions

Bits	Name	Access	Reset Value	Description
31:4	-	R	0	Reserved
3	trFunnelEmpty	R	1	Reads 1 when the trace funnel internal buffers are empty
2	-	R	0	Reserved
1	trFunnelEnable	R/W	0	Trace Funnel enabled. Setting it to 0 flushes any queued trace data to output.
0	trFunnelActive	R/W	0	Primary activate/reset bit for trace funnel

Trace Funnel Implementation Register

The Trace Funnel Implementation Register is only implemented when external trace is enabled (C_TRACE_OUTPUT = 1) and when more than one processor is connected to the MDM V (C_MB_DBG_PORTS > 1).

Reserved				CompType		VerMinor		VerMajor	
31	12	11	8	7	4	3	0		

Table 15: Trace Funnel Implementation Register Bit Definitions

Bits	Name	Access	Reset Value	Description
31:12	-	R	0	Reserved
11:8	trFunnelCompType	R	1000	Trace Funnel Component type
7:4	trFunnelVerMinor	R	0000	Trace Funnel Component Minor Version
3:0	trFunnelVerMajor	R	0001	Trace Funnel Component Major Version

PIB Sink Control Register

The PIB Sink Control Register is only implemented when external trace is enabled (C_TRACE_OUTPUT = 1).

Reserved		-	AsyncFreq		-	Calibrate		ClkCenter		Mode		Empty	-	Enable	Active
31	16	15	14	12	11	10	9	8	7	4	3	2	1	0	

Table 16: PIB Sink Control Register Bit Definitions

Bits	Name	Access	Reset Value	Description
31:16	trPibDivider	R	0x0000	Timebase selection for the PIB module. Read-only zero.
15	-	R	0	Reserved
14:12	trPibAsyncFreq	R	0	Alignment synchronization packets disabled. Read-only zero.
11:10	-	R	00	Reserved
9	trPibCalibrate	R/W	0	Set this to 1 to generate a repeating calibration pattern
8	trPibClkCenter	R	0	Adjust TRC_CLK timing to the center of the bit period. Read-only zero.
7:4	trPibMode	R/W	0	Select mode for output pins. Allowed values depend on the selected trace data width: 2 - 0000, 1001 4 - 0000, 1010 8 - 0000, 1011 16 - 0000, 1100
3	trPibEmpty	R	1	Reads 1 when PIB internal buffers are empty
2	-	R	0	Reserved
1	trPibEnable	R/W	0	Enable PIB to generate output
0	trPibActive	R/W	0	Primary activate/reset bit for PIB Sink component

Trace PIB Implementation Register

The Trace PIB Implementation Register is only implemented when external trace is enabled (`C_TRACE_OUTPUT = 1`).

Reserved				CompType		VerMinor		VerMajor	
31	12	11	8	7	4	3	0		

Table 17: Trace Funnel Implementation Register Bit Definitions

Bits	Name	Access	Reset Value	Description
31:12	-	R	0	Reserved
11:8	trPibCompType	R	1010	Trace PIB Sink Component type
7:4	trPibVerMinor	R	0000	Trace PIB Sink Component Minor Version
3:0	trPibVerMajor	R	0001	Trace PIB Sink Component Major Version

MDM V Core Interrupts

If the interrupt enable register bit in the JTAG UART control register is set, the UART raises the interrupt signal in the cycle when the TX FIFO goes empty, or in every cycle where the RX FIFO has data available.

Debug Register Space

The MDM V debug registers are listed and described in the following table. These registers are accessible using the debug register access functionality via the AXI4-Lite interface or by the external debugger via JTAG using the internal or external BSCAN.

All implemented Debug Module, Core, and Trigger registers from the RISC-V External Debug Support, Version 1.0, are listed here.

Attempting to access the unimplemented Core registers `dscratch0` and `dscratch1` results in an Illegal Instruction exception in debug mode, when complete illegal instruction exceptions are enabled (MicroBlaze V parameter `C_ILLL_INSTR_EXCEPTION = 2`).

Table 18: Register Address Space

Address (hex)	Register Name	Remark
Debug Module (DM) Registers		
04	data0	
10	dmcontrol	- hartselhi = 0 - hartsello = 0-31 depending on C_MB_DBG_PORTS The bits ackunavail, setkeepalive, and clrkeepalive have no effect.
11	dmstatus	Read-only bits: - ndmresetpending = 0 - stickyunavail = 0 - impebreak = 1 - authenticated = 1 - authbusy = 0 - hasresethaltreq = 1 - confstrptrvalid = 0 - version = 3 The bits anynonexistent and allnnonexistent can only be set if the number of cores (harts) is not 1, 2, 4, 8, 16, or 32.
12	hartinfo	Read-only value 0
14	hawindowssel	Read-only value 0
15	hawindow	Size: 1 - 32 bits depending on C_MB_DBG_PORTS
16	abstractcs	Read-only fields: - progbufsize = 1 - datacount = 1
17	command	
18	abstractauto	Read-only value 0
19	confstrptr0	Read-only value 0
1d	nextdm	Read-only value 0
20	progbuf0	
38	sbcsc	Exists when C_DBG_MEM_ACCESS > 0
39	sbaddress0	Exists when C_DBG_MEM_ACCESS > 0
3c	sbdata0	Exists when C_DBG_MEM_ACCESS > 0
40	haltsum0	
70	custom0	Exists when C_USE_UART > 0
71	custom1	Exists when C_USE_UART > 0

Table 18: Register Address Space (cont'd)

Address (hex)	Register Name	Remark
Core Registers		
7b0	dcsr	Read-only fields: • debugver = 4 • stopcount = 1 • stoptime = 1 • mprven = 0 when MicroBlaze V parameter C_USE_MMU = 0, otherwise 1 • prv = 3 when MicroBlaze V parameter C_USE_MMU = 0 The field prv and the bit ebreaku are implemented when the MicroBlaze V parameter C_USE_MMU > 0. The bit ebreaks is implemented when the MicroBlaze V parameter C_USE_MMU > 1. The bits ebreakm, stepie, cause, and step are always implemented.
7b1	dpc	
Trigger Registers		
7a0	tselect	0 - 15: 8 breakpoints, 4 write triggers, 4 read triggers
7a1	tdata1	• type = 0, 2 • dmode = 1 The u bit is implemented when the MicroBlaze V parameter C_USE_MMU > 0.
7a2	tdata2	Instruction or data access address
7a4	tinfo	Read only value 0005 (hex)

UART Read Write Byte Register

This register is read-write and accessed with the custom0 Debug Module register at hexadecimal address 70.

Reserved		Read Write Byte	
31	8	7	0

Table 20: UART Read Write Byte Register Bit Definitions

Bits	Name	Access	Reset Value	Description
31:8	-	N/A	0	Reserved
7:0	Read Write Byte	RW	0	UART Receive and Transmit Character

The register is used to read an 8-bit character, written by a program running on MicroBlaze V via standard output. This character is normally transferred via the MDM V and printed on the host computer console. This register should only be read when the UART Control Status Register TX Data Present bit is set.

The register is also used to write an 8-bit character, to be read by a program running on MicroBlaze V via standard input. This character is normally typed by the user on the host computer console, and transferred via the MDM V. This register should not be written when the UART Control Status Register RX Buffer Full bit is set.

UART Control Status Register

This register is read-write and accessed with the custom1 Debug Module register at hexadecimal address 71.

Reserved				UART_STATUS			
31		4	3				0

Table 22: UART Status Register Bit Definitions - Read

Bits	Name	Access	Reset Value	Description
31-4	Reserved	N/A	0	Reserved
3	RX Buffer Full	R	0	Receive buffer full – Do not write data to the UART Read Write Byte register
2	RX Data Not Present	R	1	Receive buffer empty – It is possible to write up to 16 characters to the UART Read Write Byte register.
1	TX Buffer Full	R	0	Transmit buffer full – It is possible to read 16 characters from the UART Read Write Byte register.
0	TX Data Present	R	0	Transmit buffer has data – It is possible to read a single character from the UART Read Write Byte register.

Reserved				TX Buffered			
31		1					0

Table 24: UART Control Register Bit Definitions - Write

Bits	Name	Access	Reset Value	Description
31-1	Reserved	N/A	0	Reserved
0	TX Buffered	W	0	When set to 1, characters written by MicroBlaze V are not discarded

Designing with the Core

This section includes guidelines and additional information to facilitate designing with the core.

General Design Guidelines

Multiprocessor Designs

The MicroBlaze™ Debug Module V supports multiple MicroBlaze V cores, each representing a single RISC-V hardware thread (hart), making it possible to use one MDM V core for multiprocessor systems with up to 32 processors.

In general, when using internal BSCAN it is recommended to use a single MDM V core for all processors. The reason for this is that each additional MDM V core has to use a separate JTAG user-defined register. Because there are only four such registers available, with USER 1 usually reserved for hardware debug, it is a limited resource. Furthermore, the Xilinx System Debugger (XSDB) command line only detects the MDM V core connected to USER 2 by default, and needs to be configured to detect additional MDM V cores. (See *Vivado Design Suite User Guide: Programming and Debugging* ([UG908](#))) for details.

When using MicroBlaze V and MicroBlaze in the same design it is required to use MDM V with MicroBlaze V and MDM with MicroBlaze. To ensure that both are detected by XSDB it is recommended to either:

- Use internal BSCAN and select user-defined register USER 3 in either MDM or MDM V.
or
- Use external BSCAN on both MDM V and MDM, and connect them to an AMD Debug Bridge IP (see *Debug Bridge LogiCORE IP Product Guide* ([PG245](#)) for details).

When implementing a MicroBlaze V multiprocessor design in Vivado using Block Automation, all processors are automatically connected to the same MDM V core.

Xilinx Virtual Cable

When using external BSCAN for debugging with the Xilinx Virtual Cable (XVC), the MDM V is connected to a Debug Bridge IP core. This connection is normally automatically inferred by the AMD Vivado™ Design Suite, and in this case the MDM V BSCAN location should be configured to select EXTERNAL HIDDEN to hide the bus interface in Vivado IP Integrator. If an explicit connection needs to be made, configure the BSCAN location as EXTERNAL instead. For further details on the Debug Bridge, see the *Debug Bridge LogiCORE IP Product Guide* ([PG245](#)).

Parallel Debug

Parallel debug can be selected to improve timing in cases where timing closure is difficult to achieve with serial debug, in particular on stacked silicon interconnect (SSI) devices where the MDM V core is placed in the master super logic region (SLR), and one or more MicroBlaze V cores are placed in other regions. In this case it is suitable to configure both the MDM V and MicroBlaze V to use AXI parallel debug, which allows clocking of the debug signals between MDM and MicroBlaze V using an AXI Register Slice. See the *AXI Interconnect LogiCORE IP Product Guide* ([PG059](#)) for more information on how to use the AXI Register Slice.

When using internal BSCAN in SSI devices, the MDM V placement is constrained to the master Super Logic Region (SLR1).

Memory, Trace and Profiling Access

Memory Access

When using the RISC-V System Bus memory access, the AXI4 Master should be connected to access the system memory accessible by the connected MicroBlaze™ cores, usually through the processor cache interfaces M_AXI_DC and M_AXI_IC. Depending on the AXI interconnect topology, this might exclude peripheral I/O accessed through the processor peripheral interface M_AXI_DP. The LMB Master ports should be connected to access the LMB local memory of the corresponding processor. This can be done by adding an additional LMB slave input to the data-side LMB block RAM Interface Controller (see the *LMB BRAM Interface Controller LogiCORE IP Product Guide* ([PG112](#))).

Trace and Profiling Access

The System Bus can also be connected to the S_AXI interface of the MDM V and MicroBlaze V cores, giving access to the RISC-V N-Trace (Nexus-based Trace) and Non-intrusive Profiling register space. With External Trace, the S_AXI interface must be enabled by selecting Enable Debug Register Access from AXI.

As an alternative, the MDM V also allows the System Bus to access Trace and Profiling directly through the normal debug interface, by selecting the Embedded or External Trace Interface. This option does not require any additional connections between the MDM V and the MicroBlaze V cores.

For detailed information on Trace and Profiling, see *MicroBlaze V Processor Reference Guide* (UG1629).

Clocking

The `S_AXI_ACLK` input is only used when Debug register access is enabled, and AXI4-Lite slave interconnect is used to access the JTAG UART or External Trace components, or when BSCAN is disabled. Then it should normally be set to the same clock as the interconnect.

The `M_AXI_ACLK` input is used when System Bus Memory Access, Embedded Trace Interface or External Trace without debug register access is enabled, and AXI4 master interconnect and/or LMB master interfaces are used. Then it must be set to the same clock as the interconnect and LMB interfaces. Different clocks for AXI4 and LMB is not supported in this case.

The `TRACE_CLK` input clock is used when external trace output is selected. This clock could be generated on-chip or be derived from an off-chip source. It can be asynchronous to all other clocks. The nominal clock frequency is 200 MHz. If another clock frequency is used, the parameter `C_TRACE_CLK_FREQ_HZ` must be manually changed accordingly.

The `TRACE_CLK_OUT` output clock is a divided by two version of `TRACE_CLK`, to provide a clock that toggles on both edges of the `TRACE_DATA` and `TRACE_CTL` data and control outputs. To create a sample point at a stable point of the outputs, a 90° phase shift is nominally added to the `TRACE_CLK_OUT` clock. The phase shift can be adjusted manually with the parameter `C_TRACE_CLK_OUT_PHASE` if necessary.

Apart from the JTAG-based UART, Debug register access, and the trace output, the MDM V core is clocked from the BSCAN when it is enabled, with a clock frequency determined by the JTAG connection.

When programming a System ACE™ device, the MDM V core clock must be at least twice as fast as the System ACE tool controller clock for the ELF file to load correctly.

Resets

The `Debug_SYS_Rst` output can be used to reset the entire embedded system on the device, including all processors and peripherals. Normally it is connected to a `proc_sys_reset` IP core. The XSDB command `rst` can be used to activate the signal.

The Debug bus connecting each individual MicroBlaze V processor handled by the MDM V core, has the `Dbg_Rst` reset signal. This signal can be used to just reset an individual processor. The XSDB command `rst -processor` can be used to activate the signal for the selected target processor. The signal is not available when AXI parallel debug is selected.

The `S_AXI_ARESETN` input is only used when Debug register access is enabled, and AXI4-Lite slave interconnect is used to access the JTAG UART or External Trace components, or when BSCAN is disabled. Then it should normally be set to the same reset as the interconnect.

The `M_AXI_ARESETN` input is used when System Bus Memory Access, Embedded Trace Interface or External Trace without debug register access is enabled, and AXI4 master interconnect and/or LMB master interfaces are used. Then it must use the same reset as the interconnect. A corresponding reset for LMB must also be used.

Debug Transport Module Interface Description

The MDM V implements the serial JTAG Debug Transport Module (DTM) interface according to RISC-V External Debug Support, Version 1.0.

- The IR register is 5 bits.
- The IDCODE register is set by the parameter `C_DTM_IDCODE` with default value 0x00000093.
- The dtmcs register has idle set to 2, bits set to 11, and version set to 1. See RISC-V External Debug Support for details on this register.

In addition, the MDM V implements a parallel AXI4-Lite Debug Module Interface, where the Debug Module, Core, and Trigger register access is directly provided via the AXI4-Lite interface. Because the interface uses byte addresses and the Debug Module addresses are word-based, they are multiplied by four.

External Trace Connection

When external trace output is used the output signals must be assigned to appropriate package pins.

The RISC-V Trace Connector Specification, Version 1.0, defines two connectors:

- MIPI20 Debug and Trace Connector
- Mictor 38-bit Debug and Trace Connector

As an example, the nominal assignment for the KC-705 board when using the [AMD FMC XM105 Debug Card](#) to connect the trace signals to the Mictor connector are listed in the following table. The FMC HPC connector pins are also listed, which allows deriving the pin assignment for other boards using the same FMC connector and debug card.

Table 25: Trace Signal Pin Assignments

KC705			FMC HPC Pin Name	Mictor Pin Number
Signal	Pin	IOB		
TRACE_clk_o	C25	No	LA00_CC_P	6
TRACE_ctl	C30	Yes	LA10_N	36
TRACE_data[0]	D29	Yes	LA10_P	38
TRACE_data[1]	H27	Yes	LA03_N	28
TRACE_data[2]	G28	Yes	LA04_P	26
TRACE_data[3]	F28	Yes	LA04_N	24
TRACE_data[4]	B30	Yes	LA09_P	22
TRACE_data[5]	A30	Yes	LA09_N	20
TRACE_data[6]	D26	Yes	LA01_CC_P	18
TRACE_data[7]	C26	Yes	LA01_CC_N	16
TRACE_data[8]	G29	Yes	LA05_P	37
TRACE_data[9]	F30	Yes	LA05_N	35
TRACE_data[10]	H30	Yes	LA06_P	33
TRACE_data[11]	G30	Yes	LA06_N	31
TRACE_data[12]	E28	Yes	LA07_P	29
TRACE_data[13]	D28	Yes	LA07_N	27
TRACE_data[14]	E29	Yes	LA08_P	25
TRACE_data[15]	E30	Yes	LA08_N	23
Logic 0 = 0V	H26	No	LA03_P	30
Logic 0 = 0V	H25	No	LA02_N	32
Logic 1 = 1.8V	H24	No	LA02_P	34

The IOB column indicates pins where it is recommended to add a constraint to place the output flip-flop in the pad to minimize skew. The parameter `C_TRACE_OUTPUT` should be set to 1 and `C_TRACE_DATA_WIDTH` should be set to 16 with this pin assignment.

Design Flow Steps

This section describes customizing and generating the core, constraining the core, and the simulation, synthesis, and implementation steps that are specific to this IP core. More detailed information about the standard AMD Vivado™ design flows and the IP integrator can be found in the following Vivado Design Suite user guides:

- *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#))
- *Vivado Design Suite User Guide: Designing with IP* ([UG896](#))
- *Vivado Design Suite User Guide: Getting Started* ([UG910](#))
- *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#))

Customizing and Generating the Core

This section includes information about using AMD tools to customize and generate the core in the AMD Vivado™ Design Suite.

If you are customizing and generating the core in the Vivado IP integrator, see the *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#)) for detailed information. IP integrator might auto-compute certain configuration values when validating or generating the design. To check whether the values do change, see the description of the parameter in this chapter. To view the parameter value, run the `validate_bd_design` command in the Tcl console.

You can customize the IP for use in your design by specifying values for the various parameters associated with the IP core using the following steps:

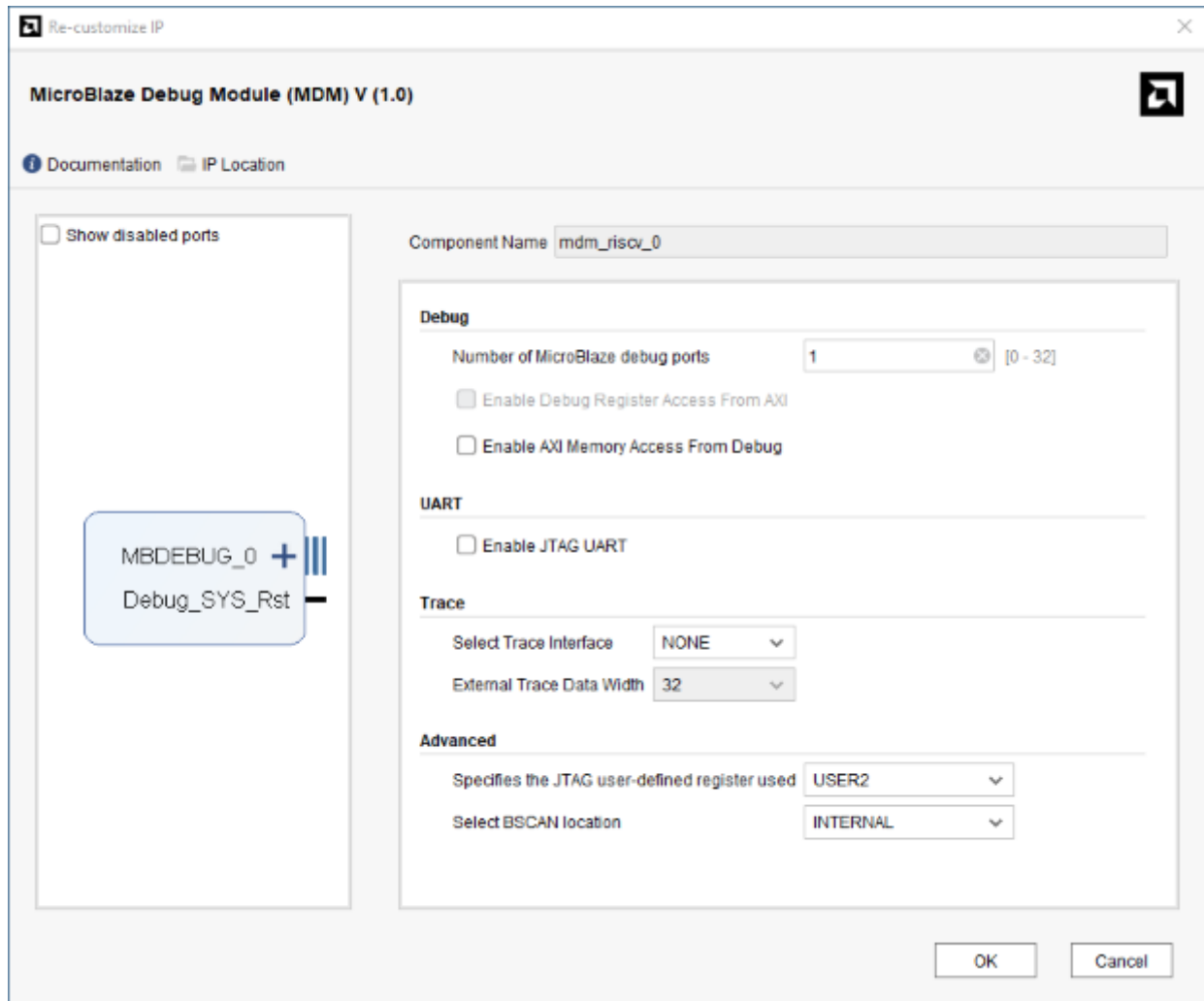
1. Select the IP from the IP catalog.
2. Double-click the selected IP or select the Customize IP command from the toolbar or right-click menu.

For details, see the *Vivado Design Suite User Guide: Designing with IP* ([UG896](#)) and the *Vivado Design Suite User Guide: Getting Started* ([UG910](#)).

Figures in this chapter are illustrations of the Vivado IDE. The layout depicted here might vary from the current version.

The Customize IP dialog box is shown in the following figure.

Figure 6: Customize IP Dialog Box



- **Number of MicroBlaze V debug ports:** Sets the number of ports available to connect to MicroBlaze™ V processors.
- **Enable Debug Register Access From AXI:** Enables the functionality to access parallel Debug or External Trace component registers from AXI and the AXI4-Lite slave interface.
- **Enable AXI Memory Access From Debug:** Enables the System Bus functionality to access memory directly from JTAG or parallel debug, using the AXI4 master interface and the LMB master interfaces.
- **Enable JTAG UART:** Enables the JTAG UART and the AXI4-Lite slave interface to access the UART registers.

- **Select Trace Interface:** Enables RISC-V System Bus Trace and Profiling register space access through the debug ports connected to the MicroBlaze™ V processors. When set to External, the TRACE interface is enabled to output data from the PIB Sink.
- **External Trace Data Width:** Defines the data width for the External trace output interface: 2, 4, 8, or 16 bits.
- **Specify the JTAG user-defined register used:** Select JTAG user-defined register. Can be set to USER1, USER2, USER3, or USER4. Should never need to be changed from USER2, unless there is a conflict with another IP core in the system.
- **Select BSCAN location:** Selects whether internal or external BSCAN is used, or disables BSCAN. Should normally be set to INTERNAL in an embedded system. Should be set to EXTERNAL or EXTERNAL HIDDEN when debugging with the Xilinx Virtual Cable (XVC) using the Debug Bridge. Can be set to NONE to disable BSCAN completely, when only Debug register access is used and no software debug is required.

Parameter Values

To tailor an MDM V core uniquely for a specific system, certain features can be parametrized in the core design. This allows you to configure a design that only uses the resources required by the system. The features that can be parametrized in MDM designs are shown in the following table.

Table 26: MDM V Design Parameters

Feature/Description	Parameter Name	Allowable Values	Default Value	VHDL Type
System Parameters				
Target family	C_FAMILY	See IP Facts	AMD Virtex™ 7	string
Debug Parameters				
Number of MicroBlaze™ debug ports	C_MB_DBG_PORTS	0–32 ¹	1	integer
Enable Debug register access from AXI	C_DBG_REG_ACCESS	0, 1	0	integer
Enable AXI Memory Access From Debug	C_DBG_MEM_ACCESS	0, 1	0	integer
UART Parameters				
Enable JTAG UART	C_USE_UART	0, 1	0	integer
Trace Parameters				
Select Trace Interface	C_TRACE_OUTPUT	0 = NONE 1 = EXTERNAL 4 = EMBEDDED	0	integer
External Trace Data Width	C_TRACE_DATA_WIDTH	2,4,8,16	0	integer
External Trace Clock Input Frequency ²	C_TRACE_CLK_FREQ_HZ	Output clock frequency multiplied by 2	200	integer
External Trace Clock Phase ²	C_TRACE_CLK_OUT_PHASE	0–360	90	integer
Advanced Parameters				
Specifies the JTAG user-defined register used	C_JTAG_CHAIN	1 = USER1 2 = USER2 3 = USER3 4 = USER4	2	integer
Select BSCAN location	C_USE_BSCAN	0 = INTERNAL 2 = EXTERNAL 3 = NONE 4 = EXTERNAL HIDDEN	0	integer
Define BSCAN id ²	C_BSCANID	0 - 0xFFFFFFFF	0x0	integer
MicroBlaze Debug connection ²	C_DEBUG_INTERFACE	0 = SERIAL 1 = PARALLEL 2 = AXI	0	integer
Use JTAG BSCAN ²	C_USE_JTAG_BSCAN	0, 1	1	integer
DTM IDCODE ²	C_DTM_IDCODE	0 - 0xFFFFFFFF	0x93	integer
Use BSCAN switch ²	C_USE_BSCAN_SWITCH	0, 1	0	integer

Table 26: MDM V Design Parameters (cont'd)

Feature/Description	Parameter Name	Allowable Values	Default Value	VHDL Type
Avoid SRL16 and SRL32 FPGA primitives ²	C_AVOID_PRIMITIVES	0,1	0	integer

Notes:

1. A value of 0 is legal and can be used if the MDM core is set up to only access AXI memory
2. Not available in the Customize IP dialog box

In addition to the parameters listed in this table, there are also parameters that are inferred for each AXI interface in the tools. Through the design, these inferred parameters control the behavior of the AXI Interconnect. For a complete list of the interconnect settings related to the AXI interface, see the *AXI Interconnect LogiCORE IP Product Guide* ([PG059](#)).

User Parameters

The following table shows the relationship between the fields in the AMD Vivado™ IDE and the user parameters (which can be viewed in the Tcl Console).

Table 27: Vivado IDE Parameter to User Parameter Relationship

Vivado IDE Parameter	User Parameter	Default Value
Number of MicroBlaze debug ports	C_MB_DBG_PORTS	1
Enable Debug Register Access from AXI	C_DBG_REG_ACCESS	0
Enable AXI Memory Access From Debug	C_DBG_MEM_ACCESS	0
Enable JTAG UART	C_USE_UART	0
Select Trace Interface	C_TRACE_OUTPUT	0 = NONE
External Trace Data Width	C_TRACE_DATA_WIDTH	32
Specifies the JTAG user-defined register used	C_JTAG_CHAIN	2 = USER2
Select BSCAN location	C_USE_BSCAN	0 = INTERNAL

Output Generation

For details, see the *Vivado Design Suite User Guide: Designing with IP* ([UG896](#)).

Constraining the Core

Required Constraints

This section is not applicable for this IP core.

Device, Package, and Speed Grade Selections

This section is not applicable for this IP core.

Clock Frequencies

This section is not applicable for this IP core.

Clock Management

The MDM V Debug logic is fully synchronous with the BSCAN module. Internally BUFG primitives are instantiated to buffer the DRCK clock from the BSCAN module.

The MDM V Debug register access is fully synchronous to the AXI4-Lite slave bus interface clock, and isolated from the BSCAN clock region with FIFOs and synchronization flip-flops when BSCAN is used.

When MicroBlaze V parallel debug interface is selected (`C_DEBUG_INTERFACE > 0`) all Debug register access is fully synchronous to the AXI4-Lite slave bus interface clock. The clocking of the connected processors depends on the used interface:

- In case `C_DEBUG_INTERFACE` is set to 1 (PARALLEL), the same clock must be used to clock the AXI4-Lite slave bus interface and all connected MicroBlaze V processors.
- In case `C_DEBUG_INTERFACE` is set to 2 (AXI), different clocks can be used by the MDM V and the connected MicroBlaze V processors, because the AXI interconnect provides clock conversion.

Clock Placement

This section is not applicable for this IP core.

Banking

This section is not applicable for this IP core.

Transceiver Placement

This section is not applicable for this IP core.

I/O Standard and Placement

This section is not applicable for this IP core.

Simulation

For comprehensive information about AMD Vivado™ simulation components, as well as information about using supported third-party tools, see the *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#)).

Synthesis and Implementation

For details about synthesis and implementation, see the *Vivado Design Suite User Guide: Designing with IP* ([UG896](#)).

Verification, Compliance, and Interoperability

Verification

The MDM V has been verified with the AMD hardware server.

Compliance Testing

No compliance testing has been performed for the MDM V.

Interoperability

The MDM V is designed to be interoperable with any third-party debugger that adheres to RISC-V External Debug Support, Version 1.0.

Upgrading

This appendix is not applicable for the first release of the core.

Debugging

This appendix includes details about resources available on the AMD Support website and debugging tools.

Finding Help with AMD Adaptive Computing Solutions

To help in the design and debug process when using the core, the [Support web page](#) contains key resources such as product documentation, release notes, answer records, information about known issues, and links for obtaining further product support. The [Community Forums](#) are also available where members can learn, participate, share, and ask questions about AMD Adaptive Computing solutions.

Documentation

This product guide is the main document associated with the core. This guide, along with documentation related to all products that aid in the design process, can be found on the [AMD Adaptive Support web page](#) or by using the AMD Adaptive Computing Documentation Navigator. Download the Documentation Navigator from the [Downloads page](#). For more information about this tool and the features available, open the online help after installation.

Answer Records

Answer Records include information about commonly encountered problems, helpful information on how to resolve these problems, and any known issues with an AMD Adaptive Computing product. Answer Records are created and maintained daily to ensure that users have access to the most accurate information available.

Answer Records for this core can be located by using the Search Support box on the main [AMD Adaptive Support web page](#). To maximize your search results, use keywords such as:

- Product name
- Tool message(s)

- Summary of the issue encountered

A filter search is available after results are returned to further target the results.

Master Answer Record for the Core

AR: [36210](#)

Technical Support

AMD Adaptive Computing provides technical support on the [Community Forums](#) for this AMD LogiCORE™ IP product when used as described in the product documentation. AMD Adaptive Computing cannot guarantee timing, functionality, or support if you do any of the following:

- Implement the solution in devices that are not defined in the documentation.
- Customize the solution beyond that allowed in the product documentation.
- Change any section of the design labeled DO NOT MODIFY.

To ask questions, navigate to the [Community Forums](#).

Debug Tools

There are many tools available to address MDM V design issues. It is important to know which tools are useful for debugging various situations.

Vivado Design Suite Debug Feature

The AMD Vivado™ Design Suite debug feature inserts logic analyzer and virtual I/O cores directly into your design. The debug feature also allows you to set trigger conditions to capture application and integrated block port signals in hardware. Captured signals can then be analyzed. This feature in the Vivado IDE is used for logic debugging and validation of a design running in AMD devices.

The Vivado logic analyzer is used to interact with the logic debug LogiCORE IP cores, including:

- ILA 2.0 (and later versions)
- VIO 2.0 (and later versions)

See the *Vivado Design Suite User Guide: Programming and Debugging* ([UG908](#)).

Simulation Debug

The simulation debug flow for Mentor Graphics Questa Advanced Simulator is illustrated in the following list. A similar approach can be used with other simulators.

- Check for the latest supported versions of Questa Advanced Simulator in the AMD Design Tools: Release Notes Guide. Is this version being used? If not, update to this version.
- If using Verilog, do you have a mixed-mode simulation license? If not, obtain a mixed-mode license.
- Ensure that the proper libraries are compiled and mapped. In the AMD Vivado™ Design Suite Flow > Simulation Settings can be used to define the libraries.
- Have you associated the intended software program for the MicroBlaze™ V processor with the simulation? Use the command Tools > Associate ELF Files in Vivado Design Suite.
- When observing the traffic on the interfaces connected to the MDM core, see the timing in the relevant specification:
 - For AXI4 and AXI4-Lite, see the [AMBA AXI and ACE Protocol Specification](#).

Hardware Debug

Hardware issues can range from link bring-up to problems seen after hours of testing. This section provides debug steps for common issues. The AMD Vivado™ debug feature is a valuable resource to use in hardware debug. The signal names mentioned in the following individual sections can be probed using the debug feature for debugging the specific problems.

General Checks

Ensure that all the timing constraints for the core were properly incorporated from the example design and that all constraints were met during implementation.

- Does it work in post-place and route timing simulation? If problems are seen in hardware but not in timing simulation, this could indicate a PCB issue. Ensure that all clock sources are active and clean.
- If using MMCMs in the design, ensure that all MMCMs have obtained lock by monitoring the `locked` port.
- If your outputs go to 0, check your licensing.

Interface Debug

AXI4-Lite Interfaces

Read from a register that does not have all 0s as a default to verify that the interface is functional. Output `S_AXI_ARREADY` asserts when the read address is valid, and output `S_AXI_RVALID` asserts when the read data/response is valid. If the interface is unresponsive, ensure that the following conditions are met:

- The `S_AXI_ACLK` input is connected and toggling.
- The interface is not being held in reset, and `S_AXI_ARESETN` is an active-Low reset
- If the simulation has been run, verify in simulation and/or a debug feature capture that the waveform is correct for accessing the AXI4-Lite interface.

Application Software Development

Device Drivers

The MDM JTAG-based UART interface is supported by the UART Lite driver, included with the AMD Vitis™ software platform.

Additional Resources and Legal Notices

Finding Additional Documentation

Technical Information Portal

The AMD Technical Information Portal is an online tool that provides robust search and navigation for documentation using your web browser. To access the Technical Information Portal, go to <https://docs.amd.com>.

Documentation Navigator

Documentation Navigator (DocNav) is an installed tool that provides access to AMD Adaptive Computing documents, videos, and support resources, which you can filter and search to find information. To open DocNav:

- From the AMD Vivado™ IDE, select **Help → Documentation and Tutorials**.
- On Windows, click the **Start** button and select **Xilinx Design Tools → DocNav**.
- At the Linux command prompt, enter `docnav`.

Note: For more information on DocNav, refer to the *Documentation Navigator User Guide* ([UG968](#)).

Design Hubs

AMD Design Hubs provide links to documentation organized by design tasks and other topics, which you can use to learn key concepts and address frequently asked questions. To access the Design Hubs:

- In DocNav, click the **Design Hubs View** tab.
- Go to the [Design Hubs](#) web page.

Support Resources

For support resources such as Answers, Documentation, Downloads, and Forums, see [Support](#).

References

These documents provide supplemental material useful with this guide:

1. *AMBA AXI and ACE Protocol Specification* ([ARM IHI0022E](#))
2. *LMB BRAM Interface Controller LogiCORE IP Product Guide* ([PG112](#))
3. *AXI UART Lite LogiCORE IP Product Guide* ([PG142](#))
4. *Vivado Design Suite User Guide: Release Notes, Installation, and Licensing* ([UG973](#))
5. *BSCAN to JTAG Converter LogiCORE IP Product Guide* ([PG365](#))
6. *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#))
7. *Vivado Design Suite User Guide: Designing with IP* ([UG896](#))
8. *Vivado Design Suite User Guide: Getting Started* ([UG910](#))
9. *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#))
10. *AXI Interconnect LogiCORE IP Product Guide* ([PG059](#))
11. *Vivado Design Suite User Guide: Programming and Debugging* ([UG908](#))
12. *Debug Bridge LogiCORE IP Product Guide* ([PG245](#))
13. *Control, Interface and Processing System LogiCORE IP Product Guide* ([PG352](#))
14. *Vivado Design Suite User Guide: Getting Started* ([UG910](#))
15. *MicroBlaze Debug Module (MDM) LogiCORE IP Product Guide* ([PG115](#))
16. [RISC-V External Debug Support, Version 1.0](#)
17. [The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA](#)
18. [The RISC-V Instruction Set Manual, Volume II: Privileged Architecture](#)
19. [RISC-V N-Trace \(Nexus-based Trace\) Specification](#)
20. [RISC-V Trace Control Interface Specification](#)
21. [RISC-V Trace Connectors Specification](#)
22. *MicroBlaze V Processor Reference Guide* ([UG1629](#))

Revision History

The following table shows the revision history for this document.

Section	Revision Summary
06/11/2025 Version 1.0	
General updates	Added support for external trace.
11/13/2024 Version 1.0	
General updates	Added support for JTAG UART and trace interface connection.
05/30/2024 Version 1.0	
Initial release	N/A

Please Read: Important Legal Notices

The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions, and typographical errors. The information contained herein is subject to change and may be rendered inaccurate for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product releases, product differences between differing manufacturers, software changes, BIOS flashes, firmware upgrades, or the like. Any computer system has risks of security vulnerabilities that cannot be completely prevented or mitigated. AMD assumes no obligation to update or otherwise correct or revise this information. However, AMD reserves the right to revise this information and to make changes from time to time to the content hereof without obligation of AMD to notify any person of such revisions or changes. THIS INFORMATION IS PROVIDED "AS IS." AMD MAKES NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE CONTENTS HEREOF AND ASSUMES NO RESPONSIBILITY FOR ANY INACCURACIES, ERRORS, OR OMISSIONS THAT MAY APPEAR IN THIS INFORMATION. AMD SPECIFICALLY DISCLAIMS ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL AMD BE LIABLE TO ANY PERSON FOR ANY RELIANCE, DIRECT, INDIRECT, SPECIAL, OR OTHER CONSEQUENTIAL DAMAGES ARISING FROM THE USE OF ANY INFORMATION CONTAINED HEREIN, EVEN IF AMD IS EXPRESSLY ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

AUTOMOTIVE APPLICATIONS DISCLAIMER

AUTOMOTIVE PRODUCTS (IDENTIFIED AS "XA" IN THE PART NUMBER) ARE NOT WARRANTED FOR USE IN THE DEPLOYMENT OF AIRBAGS OR FOR USE IN APPLICATIONS THAT AFFECT CONTROL OF A VEHICLE ("SAFETY APPLICATION") UNLESS THERE IS A SAFETY CONCEPT OR REDUNDANCY FEATURE CONSISTENT WITH THE ISO 26262 AUTOMOTIVE SAFETY STANDARD ("SAFETY DESIGN"). CUSTOMER SHALL, PRIOR TO USING

OR DISTRIBUTING ANY SYSTEMS THAT INCORPORATE PRODUCTS, THOROUGHLY TEST SUCH SYSTEMS FOR SAFETY PURPOSES. USE OF PRODUCTS IN A SAFETY APPLICATION WITHOUT A SAFETY DESIGN IS FULLY AT THE RISK OF CUSTOMER, SUBJECT ONLY TO APPLICABLE LAWS AND REGULATIONS GOVERNING LIMITATIONS ON PRODUCT LIABILITY.

Copyright

© Copyright 2024-2025 Advanced Micro Devices, Inc. AMD, the AMD Arrow logo, UltraScale, UltraScale+, Versal, Virtex, Vitis, Vivado, Zynq, and combinations thereof are trademarks of Advanced Micro Devices, Inc. Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies. AMBA, AMBA Designer, Arm, ARM1176JZ-S, CoreSight, Cortex, PrimeCell, Mali, and MPCore are trademarks of Arm Limited in the US and/or elsewhere.